

# A Neural Network for Linear Matrix Inequality Problems

Chun-Liang Lin, Chi-Chih Lai, and Teng-Hsien Huang

**Abstract**—Gradient-type Hopfield networks have been widely used in optimization problems solving. This paper presents a novel application by developing a matrix oriented gradient approach to solve a class of linear matrix inequalities (LMIs), which are commonly encountered in the robust control system analysis and design. The solution process is parallel and distributed in neural computation. The proposed networks are proven to be stable in the large. Representative LMIs such as generalized Lyapunov matrix inequalities, simultaneous Lyapunov matrix inequalities, and algebraic Riccati matrix inequalities are considered. Several examples are provided to demonstrate the proposed results. To verify the proposed control scheme in real-time applications, a high-speed digital signal processor is used to emulate the neural-net-based control scheme.

**Index Terms**—Gradient-type neural network, linear matrix inequality (LMI), quadratic stability, robust control.

## I. INTRODUCTION

LINEAR matrix inequalities (LMIs) are becoming basic tools in control as Lyapunov and Riccati equations became basic tools in the 1960s. They have emerged as a powerful formulation and design technique for a variety of linear control problems [1]. Construction for design objectives in systems and control theory can usually be cast or recast as semidefinite programming problems, i.e., LMI problems, for example, robustness analysis and robust controller design [10], [11], [13], gain scheduled controller design [14], design with pole placement constraints [4], stability analysis of fuzzy control system [19], etc. See [3] for an extensive research survey. Due to recent advances in convex optimization, efficient algorithms exist for solving LMIs; see [3] and [16] for excellent surveys of these methods and [7] for a MATLAB toolbox. It turns out that the analysis problem can be solved using numerical algorithms.

Traditional algorithms solving for LMIs perform sequentially numerical computations. For practical control applications, when the solution is to be obtained within a time of the order of 100 ns, a digital computer performing numerical computation may not comply with the desired computation time. In recent years, intensive investigation activity has been devoted to the application of artificial neural networks for solving the matrix equations [5], [17], [18], [20] and linear programming problems [6], [21] due to their parallel and distributed nature. In [18], stability and solvability analyzes of a recurrent neural network

have been investigated. Because of the parallel distributed nature of the neural networks, and advances in very large scale integration (VLSI) technology make it possible to fabricate microelectronic networks of high complexity, thus they can be available computational models for the practical control purpose [5], [22].

The presented approach constitutes a solution to the LMI problems by applying a matrix oriented gradient-type neural network. Much research has been reported for solving systems of linear equations and related problems with artificial neural networks. However, there is little literature focusing on neural solutions to the Riccati matrix equations or related equations [13]. In particular, it was rarely seen that one solves LMIs based on neural network processing [12]. Throughout this paper, an emphasis is placed on synthesizing a variety of recurrent neural networks to solve LMIs. The algorithm involves four important extensions beyond traditional numerical methods. First, it takes advantage of the special structure of the matrix inequalities, e.g., Lyapunov or Riccati. Second, the neural dynamics enables us to solve many optimization problems using the massively parallel operations of the computing units and due to the better convergence properties in comparison with iterative schemes. Third, the neuro dynamical system on the basis of differential equations usually exhibits more robustness to certain variations and it tends to retain information better through time. Finally, it possesses the potential to be used in robust adaptive control design for time-varying or gain-scheduled systems. We consider the following problems in this paper:

- Lyapunov matrix inequalities for stability and performance analysis of linear differential inclusion;
- joint synthesis of state feedback and Lyapunov or Riccati inequalities for linear differential inclusions.

As limited but representative LMIs are studied here, however, the proposed approach can be directly extended to solve other types of LMIs.

## II. NEURAL DYNAMIC EQUATION

Formulation of LMI based on Lyapunov's methods will be our main focus. Advantages of using the methods are threefold. First, many problems from Lyapunov theory can be cast as convex or quasiconvex problems. Second, the method can just as well be used to find bounds on system performance provided one does not insist on analytic solutions. Third, the neural network solving for Lyapunov equations may be realized using hardware. These features make Lyapunov's method useful in control applications.

Manuscript received May 3, 1999; revised November 5, 1999 and April 1, 2000. This work was supported by the National Science Council, Taiwan, R.O.C., under Grant NSC88-2213-E035-036.

The authors are with the Department of Automatic Control Engineering, Feng Chia University, Taichung, Taiwan, 40724 R.O.C.

Publisher Item Identifier S 1045-9227(00)06485-7.

The class of generalized Lyapunov inequalities are commonly used for determining a robust stabilizing control law. The LMI is described by

$$L(P) = BPA + A^T P B^T + D < 0, \quad P > 0 \quad (1)$$

where  $A, B, D = D^T > 0$  are given matrices of appropriate sizes and  $P = P^T \in \mathbb{R}^{m \times m}$  is the variable. The inequality symbol “<” (“>”) in (1) means that  $L(\cdot)(P)$  is negative definite (positive definite), i.e.,  $u^T L(P)u < 0$  ( $P > 0$ ),  $\forall u \neq 0$ ,  $u \in \mathbb{R}^n$ . If  $L(P)$  is simplified as  $L(P) = PA + A^T P$ , then it is well known from the linear system theory that this LMI is feasible (i.e., the LMI will be solvable and result in  $P > 0$ ), if and only if, the matrix  $A$  is asymptotically stable.

To simplify the representation let us only consider the general form

$$L(P) < 0, \quad P > 0. \quad (2)$$

To solve for LMIs in terms of recurrent neural networks, we first impose two slack matrices converting the LMI into the linear matrix equalities of the forms

$$G_1(P, R_1) = L(P) + \tilde{R}_1 \tilde{R}_1^T = 0 \quad (3a)$$

$$G_2(P, R_2) = P - \tilde{R}_2 \tilde{R}_2^T = 0 \quad (3b)$$

where  $P$  is named as the solution matrix,  $G_1 \in \mathbb{R}^{n \times n}$  and  $G_2 \in \mathbb{R}^{m \times m}$  are the objective matrices, the slack matrices  $\tilde{R}_1$  and  $\tilde{R}_2$  are restricted to be nonsingular positive definite as the following forms:

$$\begin{aligned} & \tilde{R}_1 \\ & = H_1(R_1) \\ & = \begin{bmatrix} h_{11}(r_{1,11}) & 0 & 0 & \dots & 0 \\ r_{1,21} & h_{12}(r_{1,22}) & 0 & \dots & 0 \\ r_{1,31} & r_{1,32} & h_{13}(r_{1,33}) & \ddots & 0 \\ \vdots & \vdots & \vdots & \ddots & 0 \\ r_{1,n1} & r_{1,n2} & r_{1,n3} & \dots & h_{1n}(r_{1,nn}) \end{bmatrix} \end{aligned} \quad (4a)$$

$$\begin{aligned} & \tilde{R}_2 \\ & = H_2(R_2) \\ & = \begin{bmatrix} h_{21}(r_{2,11}) & 0 & 0 & \dots & 0 \\ r_{2,21} & h_{22}(r_{2,22}) & 0 & \dots & 0 \\ r_{2,31} & r_{2,32} & h_{23}(r_{2,33}) & \ddots & 0 \\ \vdots & \vdots & \vdots & \ddots & 0 \\ r_{2,m1} & r_{2,m2} & r_{2,m3} & \dots & h_{2n}(r_{2,mm}) \end{bmatrix} \end{aligned} \quad (4b)$$

with  $R_1 = [r_{1,ij}]_{n \times n}$ ,  $R_2 = [r_{2,ij}]_{m \times m}$ ,  $\tilde{r}_{s,jj} = h_s(r_{s,jj}) > 0$ ,  $\forall s, j$ . The existence of the decomposition  $R_s \tilde{R}_s^T$  for a positive definite matrix is ensured by the elementary matrix theory [15].

**Lemma 1:** A principal submatrix of a positive definite matrix is also positive definite.

**Lemma 2:** The matrix  $A$  is positive definite, if and only if, there is a unique lower triangular matrix  $\tilde{R}$  with positive diagonal elements such that  $A = \tilde{R} \tilde{R}^T$ .

It follows from Lemma 1 that the matrices  $\tilde{R}_1$  and  $\tilde{R}_2$  defined in (4) are positive definite. Lemma 2 is known as the Cholesky decomposition. It ensures that the LMI in (2) can be expressed in the form of (3).

The next step is to establish a convex computation energy function

$$\begin{aligned} E[G(P, R_1, R_2)] &= \sum_{i=1}^n \sum_{j=1}^n c_{1,ij} [g_{1,ij}(P, R_1)] \\ &+ \sum_{i=1}^m \sum_{j=1}^m c_{2,ij} [g_{2,ij}(P, R_2)] \end{aligned} \quad (5)$$

where  $c_{s,ij}$  is the convex error function bounded below, it measures the degree of constraint violation of elements  $g_{s,ij}(P, R_s)$ . Let us define  $f_{s,ij}(g_{s,ij}) = (dc_{s,ij}/dg_{s,ij})$ . Suppose further that for all  $s, i$ , and  $j$

- 1)  $f_{s,ij}(g_{s,ij})$  is continuous or at least piecewise differentiable;
- 2)  $(df_{s,ij}(g_{s,ij})/dg_{s,ij}) \geq 0$  for  $g_{s,ij} \in (-\infty, \infty)$ ;
- 3)  $f_{s,ij}(0) = 0$ .

Solving for the LMI now becomes a constrained optimization problem. The derivation of the energy function enables us to transform the minimization problem into a set of ordinary differential equations based on neural networks with appropriate synaptic weights, input excitation, bias, and nonlinear or linear activation function.

The neural dynamic equations for solving LMIs (3a) and (3b) are described by

$$\frac{dP(t)}{dt} = -\eta_p \frac{\partial E}{\partial P} = -\eta_p \Omega, \quad P(0) = P^T(0) \quad (6a)$$

$$\frac{dR_1(t)}{dt} = -\eta_{r1} \frac{\partial E}{\partial \tilde{R}_1} = -\eta_{r1} W_1, \quad R_1(0) \neq 0 \quad (6b)$$

$$\frac{dR_2(t)}{dt} = -\eta_{r2} \frac{\partial E}{\partial \tilde{R}_2} = -\eta_{r2} W_2, \quad R_2(0) \neq 0 \quad (6c)$$

$$\tilde{R}_s = H_s(R_s), \quad s = 1, 2 \quad (6d)$$

where the derivative of a scalar valued-function  $E$  with respect to a matrix is defined by

$$\frac{\partial E}{\partial P} = \left[ \frac{\partial E}{\partial p_{ij}} \right]_{m \times m}, \quad i, j = 1, \dots, m.$$

The derivatives  $(\partial E / \partial \tilde{R}_s)$ ,  $s = 1, 2$  are defined similarly. In the above,  $P(t)$  and  $R_s(t)$ ,  $s = 1, 2$  are activation state matrices of the recurrent neural network,  $\eta_p, \eta_{rs} > 0$  are the learning rates and  $\Omega = [\omega_{ij}]_{m \times m}$ ,  $W_1 = [w_{1,ij}]_{n \times n}$ ,  $W_2 = [w_{2,ij}]_{m \times m}$  with

$$\begin{aligned} \omega_{ij} &= \sum_{k=1}^n \sum_{l=1}^n \frac{\partial g_{1,kl}(P, R_1)}{\partial p_{ij}} f_{1,kl}(g_{1,kl}(P, R_1)) \\ &+ \sum_{k=1}^m \sum_{l=1}^m \frac{\partial g_{2,kl}(P, R_2)}{\partial p_{ij}} f_{2,kl}(g_{2,kl}(P, R_2)) \\ &\quad i = 1, \dots, m, j = 1, \dots, m \end{aligned} \quad (7a)$$

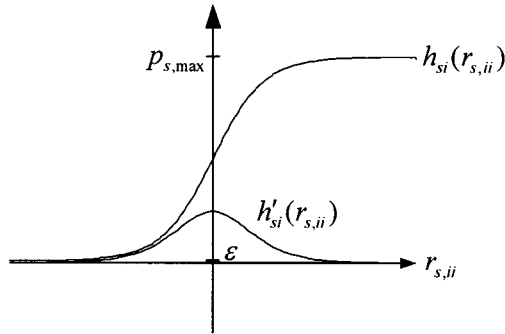


Fig. 1. A nonlinear transformation.

$$w_{1,ij} = \sum_{k=1}^n \sum_{l=1}^n \frac{\partial g_{1,kl}(P, R_1)}{\partial \tilde{r}_{1,ij}} f_{1,kl}(g_{1,kl}(P, R_1)) \quad (7b)$$

$$i = 1, \dots, n, j = 1, \dots, n$$

$$w_{2,ij} = \sum_{k=1}^m \sum_{l=1}^m \frac{\partial g_{2,kl}(P, R_2)}{\partial \tilde{r}_{2,ij}} f_{2,kl}(g_{2,kl}(P, R_2)) \quad (7c)$$

$$i = 1, \dots, m, j = 1, \dots, m$$

in which  $f_{s,kl}(g_{s,kl})$  is viewed as the activation function with the input  $g_{s,kl}$ . Note that  $e_{s,ij}(g_{s,ij})$  and  $f_{s,ij}(g_{s,ij})$  are functions of  $g_{s,ij}$  only. The constraints  $\tilde{r}_{s,ii} > 0, \forall s, i$  imposing on  $R_s(t)$  can be fulfilled by employing a limiting integrator with the continuously nonlinear transformation (see Fig. 1):

$$h_{si}(r_{s,ii}) = \varepsilon + \frac{p_{s,max} - \varepsilon}{1 + e^{-\lambda r_{s,ii}}}, \quad \forall s, i \quad (8a)$$

where the steepness factor  $\lambda > 0$  and  $\varepsilon$  is an arbitrarily small positive constant. Note that the nonlinear limiter is continuously differentiable with

$$h'_{si}(r_{s,ii}) = \frac{\lambda(p_{s,max} - \varepsilon)e^{-\lambda r_{s,ii}}}{(1 + e^{-\lambda r_{s,ii}})^2} > 0, \quad \forall s, i. \quad (8b)$$

See Fig. 1 for a graphical illustration of  $h'_{si}$ . It should be noted that the limiter's level  $p_{s,max}$  is adjustable. Adjusting  $p_{s,max}$  the neural network will result in different solutions to the LMI (2). Therefore it can be used as a design parameter in the control design. Equations (6a)–(6c) mean that the activation state matrices  $P(t)$  and  $R_s(t)$  evolve in the direction of negative gradient of  $E[G(P, R_1, R_2)]$  as time evolves. In other words, the steady activation state matrix  $\lim_{t \rightarrow \infty} P(t) = \bar{P}$  minimizes  $E[G(P, R_1, R_2)]$  in a gradient descent fashion. This is a direct matrix representation for the minimization of cost functions extending from the traditional steepest descent algorithm and can be viewed as a matrix representation of the gradient-type Hopfield network.

1) *Remark:*

- Note that the neural dynamics can be simplified further if there is in the absence of the positive definiteness constraint imposed on  $P$ . Under these situations, the objective matrix (3b) and the neural equation (6c) can be eliminated.

- If the positive definiteness constraint imposed on  $P$  is given by

$$P = P^T, \quad P > \alpha I_m, \quad \alpha > 0$$

then the objective matrix  $G_2$  can be replaced by

$$G_2(P, R_2) = P - \alpha I_m - \tilde{R}_2 \tilde{R}_2^T = 0.$$

Based on this expression one may proceed to evaluate a corresponding neural dynamic equation.

Before extending the preceding results to solve practical LMI problems, we first present some useful derivative operations of a scalar-valued function defined as (5) with respect to the solution matrix  $P$ . These equalities will be useful for putting various LMI's in the form of (6).

*Lemma 3:* Suppose that the energy function  $E = E[G(X)]$ , the solution matrix  $X \in \mathbb{R}^{m \times n}$  and the nondecreasing activation matrix  $F = [f_{ij}(g_{ij})]$ , then the following derivatives hold:

- $G(X) = AX, A \in \mathbb{R}^{l \times m}$  then  $(\partial E / \partial X) = A^T F, F \in \mathbb{R}^{l \times n}$ .
- $G(X) = AX^T, A \in \mathbb{R}^{l \times n}$  then  $(\partial E / \partial X) = F^T A, F \in \mathbb{R}^{l \times m}$ .
- $G(X) = XA, A \in \mathbb{R}^{n \times l}$  then  $(\partial E / \partial X) = F A^T, F \in \mathbb{R}^{m \times l}$ .
- $G(X) = X^T A, A \in \mathbb{R}^{m \times l}$  then  $(\partial E / \partial X) = A F^T, F \in \mathbb{R}^{n \times l}$ .
- $G(X) = AXH^T, A \in \mathbb{R}^{n \times n}$  and  $H \in \mathbb{R}^{n \times n}$ , then  $(\partial E / \partial X) = A^T F H, F \in \mathbb{R}^{n \times n}$ .
- $G(X) = A^T X H, A \in \mathbb{R}^{n \times n}$  and  $H \in \mathbb{R}^{n \times n}$ , then  $(\partial E / \partial X) = A F H^T, F \in \mathbb{R}^{n \times n}$ .

*Proof:* See Appendix. ■

Based on (6) and Lemma 3, we can now obtain a set of neural dynamic equations solving for the LMI (1) as follows:

$$\frac{dP}{dt} = -\eta_p [B^T F_1(P, \tilde{R}_1) A^T + A F_1(P, \tilde{R}_1) B + F_2(P, \tilde{R}_2)], \quad P(0) = P^T(0) \quad (9a)$$

$$\frac{dR_1}{dt} = -\eta_{r1} F_1(P, \tilde{R}_1) \tilde{R}_1, \quad R_1(0) \neq 0 \quad (9b)$$

$$\frac{dR_2}{dt} = -\eta_{r2} F_2(P, \tilde{R}_2) \tilde{R}_2, \quad R_2(0) \neq 0 \quad (9c)$$

where the learning rates  $\eta_p, \eta_{r1}, \eta_{r2} > 0$ , and the activation matrices are

$$F_1(P, \tilde{R}_1) = F_1(BPA + A^T P B^T + D + \tilde{R}_1 \tilde{R}_1^T)$$

$$F_2(P, \tilde{R}_2) = F_2(P - \tilde{R}_2 \tilde{R}_2^T)$$

- Remark:* For the discrete Lyapunov matrix inequality described by

$$L(P) = A^T P A - P < 0, \quad P > 0$$

where  $P = P^T \in \mathbb{R}^{m \times m}$  is the variable. Suppose that  $A$  is asymptotically stable in the discrete-time sense. Therefore,  $P$  must be positive definite. The corresponding neural dynamic

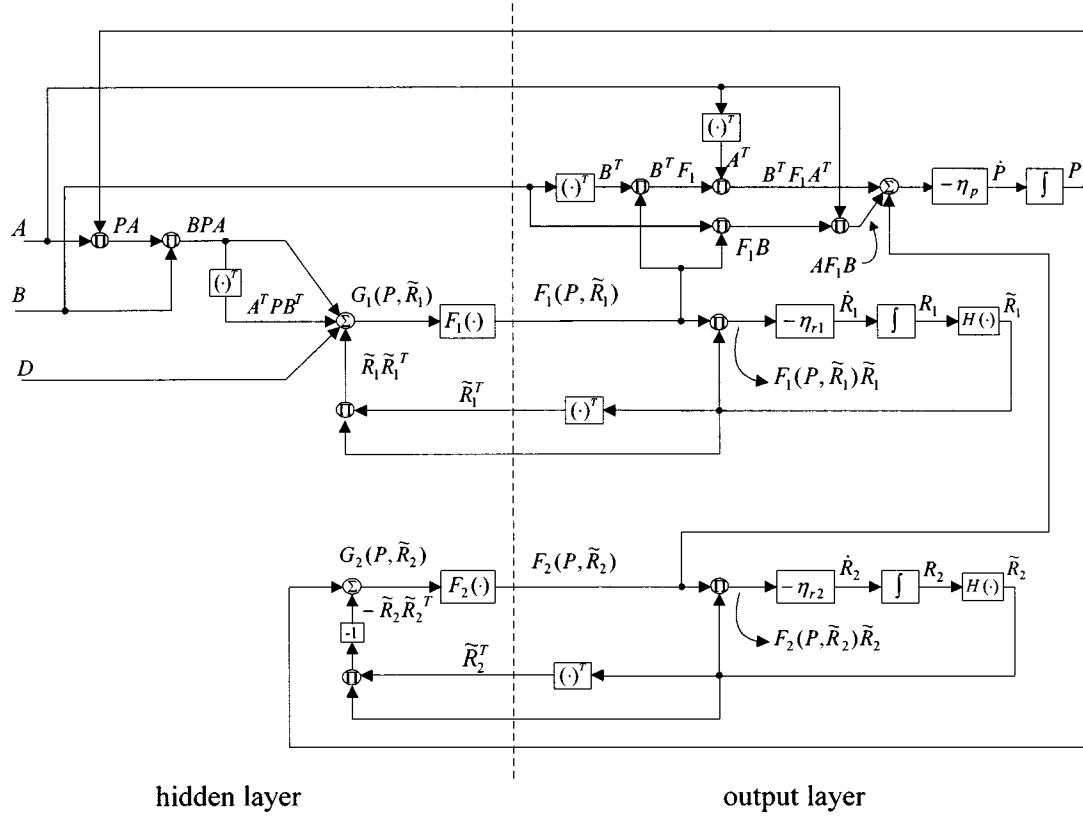


Fig. 2. Neural-network structure.

equations solving for the solution candidates are given by

$$\begin{aligned} \frac{dP}{dt} &= -\eta_p [AF(P, \tilde{R})A^T - F(P, \tilde{R})] \\ \frac{dR}{dt} &= -\eta_r F(P, \tilde{R})\tilde{R} \end{aligned}$$

where  $\tilde{R}$  is defined as in (4) and the activation matrix is

$$F(P, \tilde{R}) = F(A^T P A - P + \tilde{R}\tilde{R}^T).$$

### III. STABILITY AND SOLVABILITY

The elements of the gradient-type neural networks (6a)–(6c) can be described as follows:

$$\begin{aligned} \frac{dp_{ij}(t)}{dt} &= -\eta_p \left\{ \sum_{k=1}^n \sum_{l=1}^n \frac{\partial g_{1,kl}(P, R_1)}{\partial P_{ij}} \right. \\ &\quad \cdot \left. f_{1,ij}[g_{1,ij}(P, R_1)] \right\} \end{aligned} \quad (10a)$$

$$\begin{aligned} \frac{dr_{s,ij}(t)}{dt} &= -\eta_{rs} \left\{ \sum_k \sum_l \frac{\partial g_{s,kl}(P, R_s)}{\partial \tilde{r}_{s,ij}} \right. \\ &\quad \cdot \left. f_{s,ij}[g_{s,ij}(P, R_s)] \right\}, \quad s = 1, 2. \end{aligned} \quad (10b)$$

The architecture of the gradient-type neural network consists of three output layers and two hidden layers which are bidirectionally connected with the output layers of neurons. There is a functional transformation  $f_{s,ij}(\cdot)$  for each neuron in the hidden layer, an integral transformation for each neuron of  $p_{ij}$  in the output layer, and an integral-limiting transformation for each

neuron of  $r_{s,ij}$  in the output layer. The activation state matrix  $P(t)$  corresponds to the solution matrix. The neural-network structure for LMI solving can be illustrated as in Fig. 2.

By assumptions (A1)–(A2), then  $e_{s,ij} > -\infty$  and  $(d^2 e_{s,ij} / dg_{s,ij}^2) \geq 0$ ,  $g_{s,ij} \in (-\infty, \infty)$ ,  $\forall s, i, j$ , i.e., all  $e_{1,ij}$  and  $e_{2,ij}$  are convex and bounded from below [9]. Therefore, the sum of  $e_{1,ij}$  and  $e_{2,ij}$  forming  $E$  is also convex and bounded below. Applying “vec” operation, it is straightforward to demonstrate that the system of differential equations (6) is asymptotically stable and its steady-state represents the solution matrix via the following derivation:

$$\begin{aligned} \frac{dE[G(P, R_1, R_2)]}{dt} &= \left( \frac{\partial E[G(P, R_1, R_2)]}{\partial \text{vec}(P(t))} \right)^T \frac{d \text{vec}(P(t))}{dt} + \sum_{s=1}^2 \\ &\quad \cdot \left( \frac{\partial E[G(P, R_1, R_2)]}{\partial \text{vec}(\tilde{R}_s(t))} \right)^T \left( \frac{\partial \text{vec}(\tilde{R}_s(t))}{\partial \text{vec}(R_s(t))} \right)^T \\ &\quad \cdot \frac{d \text{vec}(R_s(t))}{dt} \\ &= \left\{ -\frac{1}{\eta_p} \left( \frac{d \text{vec}(P(t))}{dt} \right)^T \frac{d \text{vec}(P(t))}{dt} \right\} \\ &\quad + \sum_{s=1}^2 \left\{ -\frac{1}{\eta_{rs}} \left( \frac{d \text{vec}(R_s(t))}{dt} \right)^T \left( \frac{\partial \text{vec}(\tilde{R}_s(t))}{\partial \text{vec}(R_s(t))} \right)^T \right. \\ &\quad \cdot \left. \frac{d \text{vec}(R_s(t))}{dt} \right\} \end{aligned} \quad (11)$$

where the “vec” operator is defined as

$$\text{vec } P = [p_{11} \dots p_{m1} : p_{12} \dots p_{m2} : \dots : p_{1m} \dots p_{mm}]^T.$$

It is seen that the first term of the right-hand side (RHS) of (11) satisfies

$$\begin{aligned} & -\frac{1}{\eta_p} \left( \frac{d \text{vec}(P(t))}{dt} \right)^T \frac{d \text{vec}(P(t))}{dt} \\ & = -\frac{1}{\eta_p} \sum_{i=1}^m \sum_{j=1}^m \left( \frac{dp_{ij}(t)}{dt} \right)^2 \leq 0. \end{aligned} \quad (12)$$

For the second term of the RHS of (11), we have

$$\begin{aligned} & \left( \frac{\partial \text{vec}(\tilde{R}_1(t))}{\partial \text{vec}(R_1(t))} \right)^T \\ & = \begin{bmatrix} \frac{\partial h_{11}(r_{1,11})}{\partial r_{1,11}} & \frac{\partial r_{1,21}}{\partial r_{1,11}} & \dots & \frac{\partial h_{1n}(r_{1,nn})}{\partial r_{1,11}} \\ \frac{\partial h_{11}(r_{1,11})}{\partial r_{1,21}} & \frac{\partial r_{1,21}}{\partial r_{1,21}} & \dots & \frac{\partial h_{1n}(r_{1,nn})}{\partial r_{1,21}} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial h_{11}(r_{1,11})}{\partial r_{1,nn}} & \frac{\partial r_{1,21}}{\partial r_{1,nn}} & \dots & \frac{\partial h_{1n}(r_{1,nn})}{\partial r_{1,nn}} \end{bmatrix} \in \mathbb{R}^{n^2 \times n^2} \\ & = \begin{bmatrix} h'_{11}(r_{1,11}) & 0 & 0 & \dots & 0 & 0 \\ 0 & 1 & 0 & \dots & 0 & 0 \\ 0 & 0 & \ddots & \ddots & \vdots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 & \vdots \\ 0 & 0 & \dots & 0 & \ddots & 0 \\ 0 & 0 & \dots & \dots & 0 & h'_{1n}(r_{1,nn}) \end{bmatrix} \geq 0, \quad \forall t. \end{aligned}$$

Therefore, using (8b) we have

$$\begin{aligned} & -\frac{1}{\eta_{r1}} \left( \frac{d \text{vec}(R_1(t))}{dt} \right)^T \left( \frac{\partial \text{vec}(\tilde{R}_1(t))}{\partial \text{vec}(R_1(t))} \right)^T \frac{d \text{vec}(R_1(t))}{dt} \\ & = -\frac{1}{\eta_{r1}} \left[ h'_{11} \left( \frac{dr_{1,11}}{dt} \right)^2 + \left( \frac{dr_{1,21}}{dt} \right)^2 \right. \\ & \quad + \dots + \left( \frac{dr_{1,n1}}{dt} \right)^2 + h'_{12} \left( \frac{dr_{1,22}}{dt} \right)^2 \\ & \quad \left. + \dots + \left( \frac{dr_{1,n2}}{dt} \right)^2 + \dots + h'_{1n} \left( \frac{dr_{1,nn}}{dt} \right)^2 \right] \\ & \leq 0, \quad \forall t. \end{aligned} \quad (13)$$

Similarly, it can be proved that

$$-\frac{1}{\eta_{r2}} \left( \frac{d \text{vec}(R_2(t))}{dt} \right)^T \left( \frac{\partial \text{vec}(\tilde{R}_2(t))}{\partial \text{vec}(R_2(t))} \right)^T \frac{d \text{vec}(R_2(t))}{dt} \leq 0, \quad \forall t. \quad (14)$$

Combining (11)–(14), we finally attain at

$$\frac{dE[G(P(t), R_1(t), R_2(t))]}{dt} \leq 0.$$

Clearly,  $E[G(P(t), R_1(t), R_2(t))]$  is a Lyapunov function, i.e., it is a monotonic decreasing function in time. Note that to ensure the steady-state value of  $P(t)$  being positive definite, we require that  $R_1(t)$  and  $R_2(t)$  converge faster than  $P(t)$ . It is thus suggested to choose  $\eta_p \ll \eta_{rs}$ .

We now investigate solvability of the LMI (1). It can be found that the situation of  $\dot{E}[G(P(t), R_1(t), R_2(t))] = 0$  occurs as  $P$  and  $R_1$  and  $R_2$  converge. By assumption (A1) and (A3), we have  $\bar{\Gamma} = F_1(\bar{P}, \bar{R}_1) = 0$  where  $\lim_{t \rightarrow \infty} \tilde{R}_1 = \bar{R}_1$  iff  $B\bar{P}A + A^T\bar{P}B^T + D + \bar{R}_1\bar{R}_1^T = 0$  and  $\bar{\Omega} = F_2(\bar{P}, \bar{R}_2) = 0$  iff  $\bar{P} - \bar{R}_2\bar{R}_2^T = 0$  where  $\lim_{t \rightarrow \infty} \tilde{R}_2 = \bar{R}_2$ . Equations (9a)–(9c) now imply, respectively,

$$B^T\bar{\Gamma}A^T + A\bar{\Gamma}B + \bar{\Omega} = 0 \quad (15)$$

when  $(dP/dt) = 0$  (i.e.,  $P$  converges)

$$\bar{\Gamma}\bar{R}_1 = 0$$

when  $(dR_1/dt) = 0$  (i.e.,  $R_1$  converges), and

$$\bar{\Omega}\bar{R}_2 = 0$$

when  $(dR_2/dt) = 0$  (i.e.,  $R_2$  converges). Since  $\tilde{R}_1 > 0$  and  $\tilde{R}_2 > 0$ , we have  $\bar{\Gamma} = 0$  and  $\bar{\Omega} = 0$ . Matrix equality (15) can be equivalently expressed as

$$[B^T \quad A]\bar{\Gamma} \begin{bmatrix} A^T \\ B \end{bmatrix} + \bar{\Omega} = 0.$$

Clearly, the necessary and sufficient condition for (15) to hold iff  $\bar{\Gamma} = 0$  and  $\bar{\Omega} = 0$  is that the augmented matrix  $[B^T \quad A]$  is of full rank.

In the preceding derivations, the objective function  $E[G(P, R_1, R_2)]$  serves as a norm to evaluate the degree of constraint satisfaction of  $G_1(P, R_1)$  and  $G_2(P, R_2)$ . There are many potential choices for an individual component  $c_{s,ij}(g_{s,ij})$ . The selection of  $c_{s,ij}(g_{s,ij})$ , in turn, determines the form of  $f_{s,ij}(g_{s,ij})$ . The mapping  $f_{s,ij}(g_{s,ij})$  can be interpreted as an activation function of the neuron model. For example, if  $c_{s,ij}(g_{s,ij}) = \frac{1}{2}g_{s,ij}^2$  then the activation function is  $f_{s,ij}(g_{s,ij}) = g_{s,ij}$ . If  $c_{s,ij} = \cosh(g_{s,ij})$  then  $f_{s,ij}(g_{s,ij}) = \sinh(g_{s,ij})$ .

1) Remark: Currently, VLSI implementation of Hopfield-like neural networks is still a difficult problem because of the complex connectivity between a large number of processing nodes. For most cases, the bottleneck comes from the wiring of many processing nodes on a two-dimensional surface of a

silicon wafer. An effective way such as that proposed in [6] may be applied to simplify the Hopfield-type neural networks.

#### IV. NEURAL DYNAMICS FOR A CLASS OF LMIs

##### A. Simultaneous Lyapunov Inequality Problem

The problem arises in determining Lyapunov stability for linear inclusion systems of the form

$$\dot{x} = A(t)x. \quad (16)$$

The system is stable for  $A(t)$  in the convex set [2]

$$A(t) \in C_o \equiv \text{convex hull}(A_1, A_2, \dots, A_L)$$

where  $A_i \in \mathbb{R}^{n \times n}$ ,  $\forall i$  are given and stable, if, and only if, there exists a global variable  $P > 0$  such that

$$A^T P + P A < 0, \quad A \in C_o \quad (17)$$

or equivalently

$$A_i^T P + P A_i < 0, \quad i = 1, \dots, L. \quad (18)$$

Determining quadratic stability for the system is an LMI problem in the variable  $P$ . To determine the variable  $P$ , the LMI's are converted into a group of linear matrix equalities

$$A_i^T P + P A_i + \tilde{R}_i \tilde{R}_i^T = 0, \quad i = 1, \dots, L$$

where  $\tilde{R}_i$ ,  $i = 1, \dots, L$  are defined as in (4). Since  $A_i \in \mathbb{R}^{n \times n}$ ,  $\forall i$  have been assumed to be stable, thus  $P$  must be positive definite. The remaining work is to find the permissible matrix  $P$  satisfying (17).

To find the neural dynamics, the following objective matrices are defined:

$$G_i(P, R_i) = A_i^T P + P A_i + \tilde{R}_i \tilde{R}_i^T, \quad i = 1, \dots, L. \quad (19)$$

Based on these objective matrices and using Lemma 3, the neural dynamics solving for the problem can be easily formulated as follows:

$$\frac{dP}{dt} = -\eta_p \frac{\partial E}{\partial P} = -\eta_p \sum_{i=1}^L [A_i F_i(P, \tilde{R}_i) + F_i(P, \tilde{R}_i) A_i^T], \quad P(0) = P^T(0) \quad (20a)$$

$$\frac{dR_i}{dt} = -\eta_{r_i} \frac{\partial E}{\partial \tilde{R}_i} = -\eta_{r_i} F_i(P, \tilde{R}_i) \tilde{R}_i, \quad R_i(0) \neq 0, i = 1, \dots, L \quad (20b)$$

where the activation matrix is

$$F_i(P, \tilde{R}_i) = F_i(A_i^T P + P A_i + \tilde{R}_i \tilde{R}_i^T), \quad i = 1, \dots, L.$$

##### B. Stabilizing Control Problem

Consider the linear time-invariant system

$$\dot{x} = Ax + Bu \quad (21)$$

where  $A \in \mathbb{R}^{n \times n}$ ,  $B \in \mathbb{R}^{n \times m}$ . The state feedback control law is given by

$$u = Kx. \quad (22)$$

The system is said to be quadratically stabilizable if there exists a state-feedback gain  $K$  such that the closed-loop system  $\dot{x} = (A + BK)x$  is quadratically stable, or equivalently, there exists  $P = P^T > 0$  such that

$$(A + BK)^T P + P(A + BK) < 0. \quad (23)$$

An alternative equivalent condition is that, there exists  $Q = Q^T > 0$  such that

$$Q(A + BK)^T + (A + BK)Q < 0. \quad (24)$$

Define  $X = KQ$  so that  $K = XQ^{-1}$ . Substituting this into (24) yields

$$AQ + QA^T + BX + X^T B^T < 0. \quad (25)$$

The LMI includes two variables in  $Q$  and  $X$ . Thus, the closed-loop system is quadratically stabilizable if and only if there exist  $Q > 0$  and  $X$  such that the LMI (25) holds.

Using the elimination procedure for matrix variables  $Q$  and  $X$ , (25) can be equivalently expressed as

$$\tilde{B}^T (AQ + QA^T) \tilde{B} < 0 \quad (26)$$

where  $\tilde{B}$  is an orthogonal complement of  $B$ , i.e.,

$$\tilde{B}^T B = 0$$

and  $[\tilde{B} \ B]$  is nonsingular. Now using Finsler's lemma [3], (26) is also equivalent to

$$AQ + QA^T - \sigma BB^T < 0, \quad \sigma \in \mathbb{R} \quad (27)$$

where  $\sigma$  is some scalar. Clearly, the variable  $X$  has been eliminated from (25). We can always assume  $\sigma > 0$ ; since (27) is homogeneous in  $Q$  and  $\sigma$ , we may set  $\sigma = 1$  without loss of generality. If the positive definite  $Q$  satisfies (27), a stabilizing state-feedback gain is given by

$$K = -\frac{1}{2}\sigma B^T Q^{-1}. \quad (28)$$

Note that the simplified LMI (27) remains only one variable  $Q$ . To solve this equation, we first define the following objective matrix:

$$G(Q, R) = AQ + QA^T - \sigma BB^T + \tilde{R} \tilde{R}^T = 0 \quad (29)$$

where the slack matrix  $\tilde{R}$  is defined as in (4). The corresponding dynamic equations of the recurrent neural network can be ob-

tained by applying (6) and Lemma 3

$$\begin{aligned} \frac{dQ}{dt} &= -\eta_q \frac{\partial E}{\partial Q} = -\eta_q [A^T F(Q, \tilde{R}) + F(Q, \tilde{R})A], \\ Q(0) &= Q^T(0) \end{aligned} \quad (30a)$$

$$\frac{\partial R}{\partial t} = -\eta_r \frac{\partial E}{\partial \tilde{R}} = -\eta_r F(Q, \tilde{R})\tilde{R}, \quad R(0) \neq 0 \quad (30b)$$

where the activation matrix is

$$F(Q, \tilde{R}) = F(AQ + QA^T - \sigma BB^T + \tilde{R}\tilde{R}^T). \quad (30c)$$

As  $Q$  has been obtained from (30a), the control gain is recovered as  $K = -0.5\sigma B^T Q^{-1}$ . For the actual applications, however, computation for the matrix inverse  $Q^{-1}$  should be avoided. Under this consideration, the neural network given in [13] can also be used to realize the gain matrix.

### C. Algebraic Riccati Matrix Inequality Problem

The class of algebraic Riccati matrix inequalities are commonly encountered in quadratic stabilizing control for determining a robust stabilizing control law. It is typically described by

$$A^T P + PA + (PB - C^T)(D + D^T)^{-1}(B^T P - C) + Q < 0 \quad (31)$$

where  $P = P^T > 0$  is the variable,  $D + D^T > 0$ , and  $A, B, C, D, Q = Q^T$  are given matrices of appropriate sizes. This is a quadratic matrix inequality in the variable  $P$ . We assume for simplicity that  $A$  is stable and the system  $(A, B, C)$  is minimal. Using Schur theorem [8], it can be equivalently expressed as an LMI

$$\begin{bmatrix} PA + A^T P + Q & PB - C^T \\ B^T P - C & -D^T - D \end{bmatrix} < 0, \quad P > 0. \quad (32)$$

Or equivalently

$$\bar{A}P\bar{B} + \bar{B}^T P \bar{A}^T + \bar{D} < 0, \quad P > 0 \quad (33)$$

where

$$\bar{A} = \begin{bmatrix} I \\ 0 \end{bmatrix}, \quad \bar{B} = [A \ B], \quad \bar{D} = \begin{bmatrix} Q & -C^T \\ -C & -D^T - D \end{bmatrix}.$$

A Riccati equation has many solutions. However, if the equation has a symmetric positive definite solution then it is unique. Suppose that there is a positive definite solution to (31). Now to avoid the solution obtained converges to the one which is not positive definite, an additional constraint as in (3b) can be imposed

$$G_1(P, R_1) = \bar{A}P\bar{B} + \bar{B}^T P \bar{A}^T + \bar{D} + \tilde{R}_1 \tilde{R}_1^T \quad (34a)$$

$$G_2(P, R_2) = P - \tilde{R}_2 \tilde{R}_2^T. \quad (34b)$$

Applying the derivation presented in (6) and Lemma 3, the recurrent neural dynamics for solving the Riccati matrix in-

equality is given by

$$\begin{aligned} \frac{dP}{dt} &= -\eta_p [\bar{A}^T F_1(P, \tilde{R}_1) \bar{B}^T + \bar{B} F_1(P, \tilde{R}_1) \bar{A} \\ &\quad + F_2(P, \tilde{R}_2)], \quad P(0) = P^T(0) \end{aligned} \quad (35a)$$

$$\frac{dR_1}{dt} = -\eta_{r1} F_1(P, \tilde{R}_1) \tilde{R}_1, \quad R_1(0) \neq 0 \quad (35b)$$

$$\frac{dR_2}{dt} = -\eta_{r2} F_2(P, \tilde{R}_2) \tilde{R}_2, \quad R_2(0) \neq 0 \quad (35c)$$

where the activation matrices are

$$F_1(P, \tilde{R}_1) = F_1(\bar{A}P\bar{B} + \bar{B}^T P \bar{A}^T + \bar{D} + \tilde{R}_1 \tilde{R}_1^T)$$

$$F_2(P, \tilde{R}_2) = F_2(P - \tilde{R}_2 \tilde{R}_2^T).$$

## V. ILLUSTRATIVE EXAMPLES

In the following examples, elements of the convex energy function were defined as

$$e_{ij} = g_{ij} \tan^{-1}(g_{ij}) - \ln \sqrt{1 + g_{ij}^2}, \quad \forall i, j.$$

Therefore the corresponding activation functions are obtained as

$$f_{ij} = \tan^{-1}(g_{ij}), \quad \forall i, j.$$

Results of Example 1 were conducted on a hardware-in-the-loop simulation platform. Numerical algorithms solving for the neural dynamic equations in Examples 2 and 3 were written in C++ programming language and simulated on a 64-bit personal computer.

*Example 1:* Consider the perturbed system of the form

$$\begin{aligned} \dot{x} &= Ax + Bu + \tilde{\Delta}(x, u, t), \\ (A, B) &\in C_o\{(A_1, B_1), \dots, (A_L, B_L)\} \end{aligned} \quad (36)$$

where  $B_i = B(\sigma_i)$ ,  $A_i = A(\sigma_i)$ ,  $i = 1, \dots, L$  are the stable system matrices evaluated at the  $i$ th operating point  $\sigma_i$ ,  $\tilde{\Delta}(x, u, t)$  is the nonlinear time-varying perturbation satisfying the matching condition [2]

$$\tilde{\Delta}(x, u, t) = B\Delta(x, u, t)$$

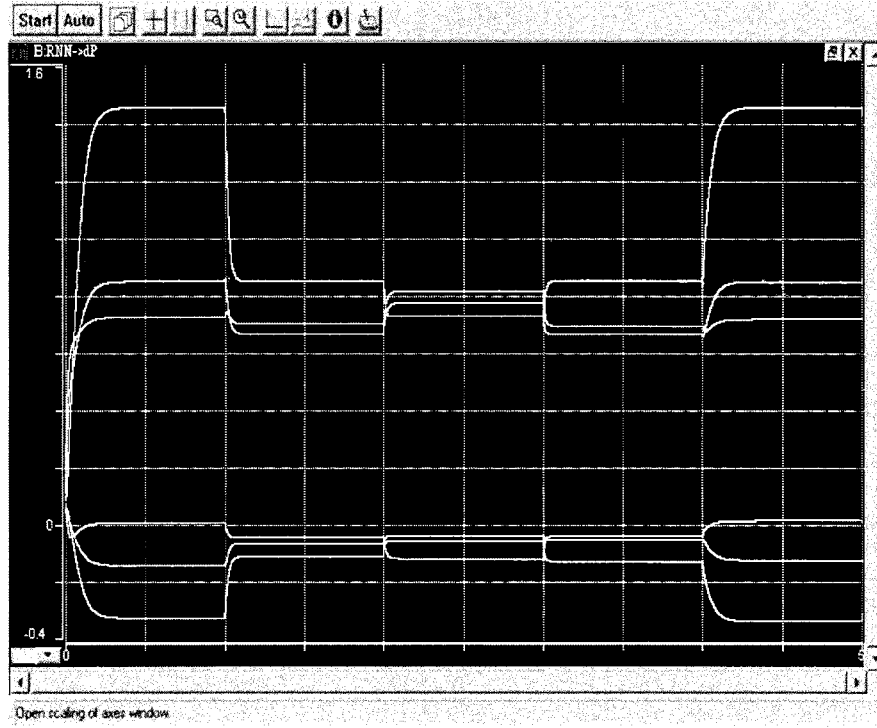
and the perturbation upper bound is estimated as

$$\|\Delta(x, u, t)\| \leq \alpha \|x\| + \beta \|u\|, \quad \beta < 1$$

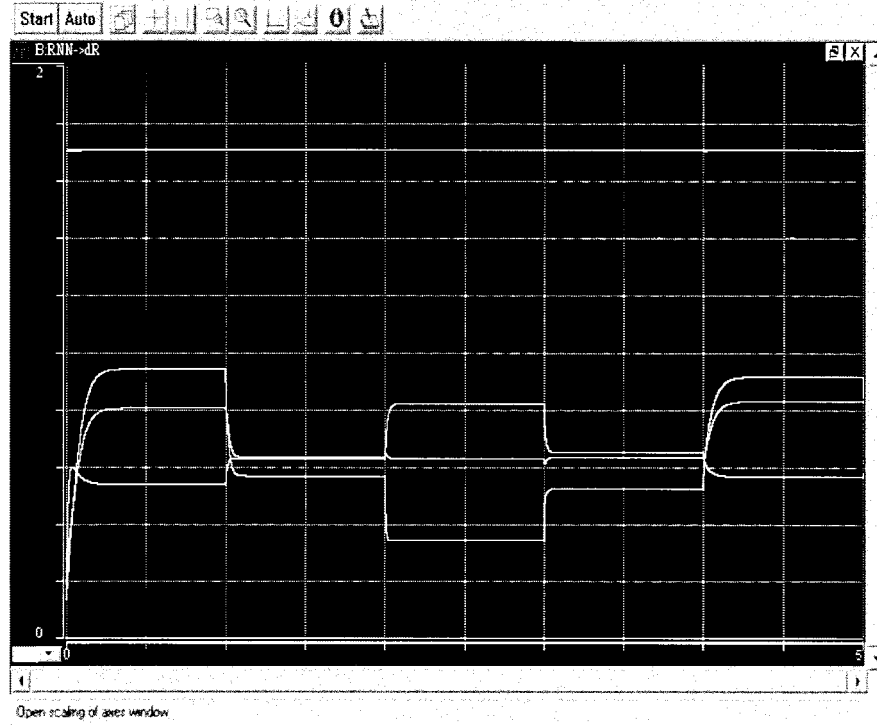
over all operating conditions. In practical applications,  $\tilde{\Delta}(x, u, t)$  can be viewed as the linearization error with  $A$  and  $B$  being the nominal system matrices. Since  $A_i$ ,  $i = 1, \dots, L$  have been assumed to be stable, the control law is mainly designed to ensure the overall stability while there is in the presence of the perturbation.

Since  $A_i$ ,  $i = 1, \dots, L$  are stable we know that there exist  $P_i > 0$ ,  $\forall i$  such that

$$P_i A_i + A_i^T P_i < 0, \quad \forall i \quad (37)$$



(a)



(b)

Fig. 3. (a) Transient of  $P$  and (b) transient of  $\tilde{R}_1$ .

The LMI's can be converted into the linear matrix equalities by imposing slack matrices  $\tilde{R}_i$

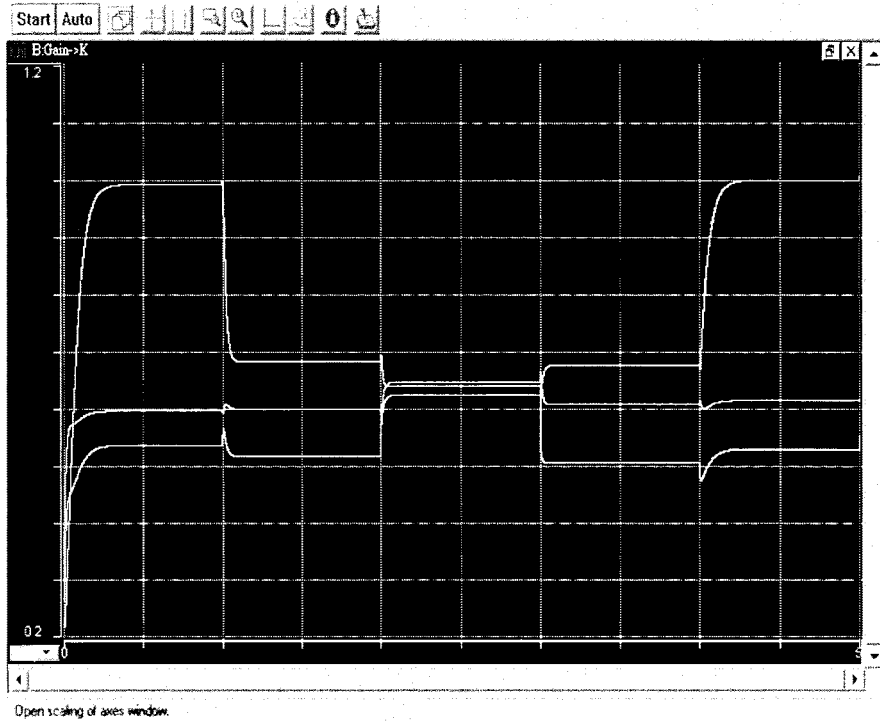
$$P_i A_i + A_i^T P_i + \tilde{R}_i \tilde{R}_i^T = 0, \quad i = 1, \dots, L. \quad (38)$$

that the perturbed system would be stable provided that the control law at the  $i$ th operating condition is given by

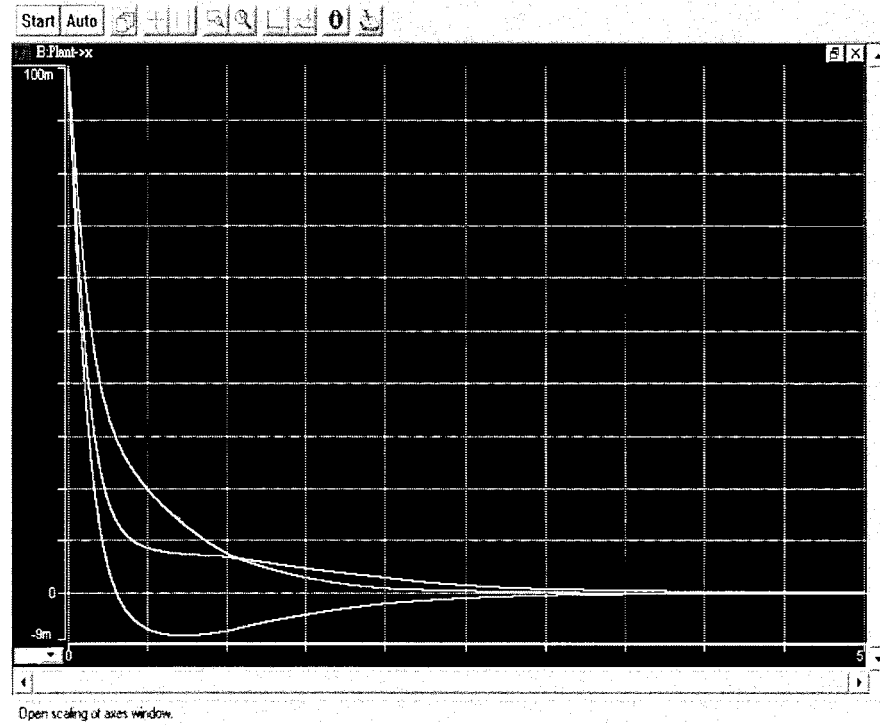
$$u = -K_i x$$

Based on the preliminary result given in [2], it can be proven where  $K_i = K(\sigma_i) = \gamma_i B^T P_i$ ,  $\gamma_i > 0$ , the design parameter





(a)



(b)

Fig. 4. (a) Transient of  $K$  and (b) transient of  $x$ .

$\gamma_i$  satisfies the following condition:

$$\gamma_i > \frac{\alpha^2}{2\lambda_{\min}(\tilde{R}_i \tilde{R}_i^T) (1 - \beta)}, \quad i = 1, \dots, L \quad (39)$$

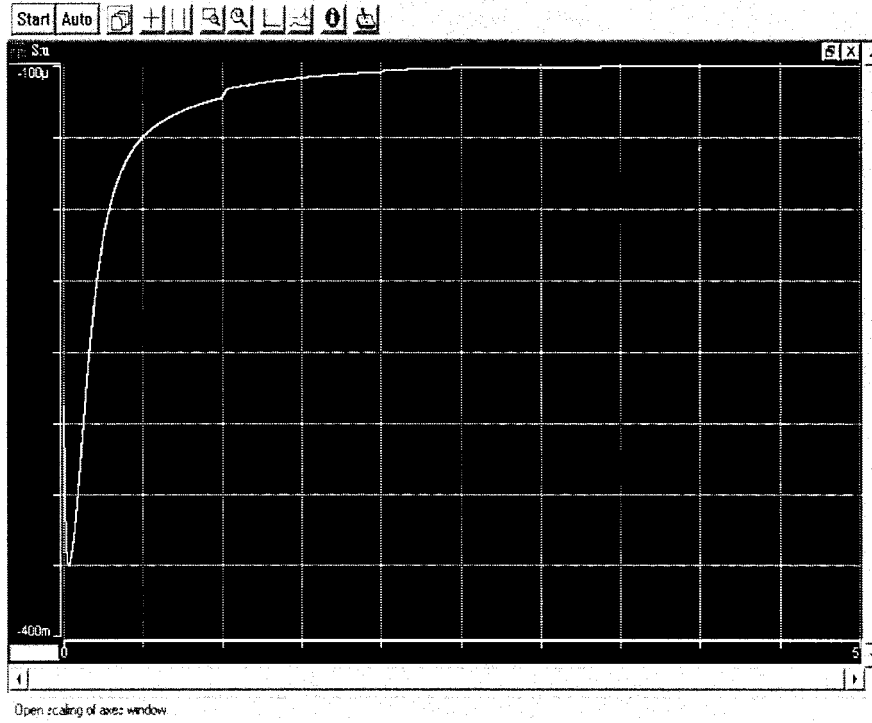
where  $\lambda_{\min}(\cdot)$  denotes the minimum eigenvalue. The control law for intermediate operating conditions are linearly interpo-

lated by computing the function  $\hat{K}(\sigma_i)$  such that

$$\hat{K}(\sigma_i) = K(\sigma_i).$$

Then the actual control law is implemented as

$$u(t) = -K(\sigma(t))x(t).$$



(c)

Fig. 4. (Continued.) (c) Transient of  $u$ .

Now consider a perturbed system of the form of (1) with the stable coefficient matrix  $A$  and vector  $B$

$$A = \begin{bmatrix} -1 - \left| \sin\left(\frac{10\pi\sigma_i}{4}\right) \right| & -1 - \left| \cos\left(\frac{10\pi\sigma_i}{4}\right) \right| & -2 \\ 0 & -2 - \left| \cos\left(\frac{10\pi\sigma_i}{4}\right) \right| & -3 \\ 0 & 0 & -3 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 \\ 1 \\ 1 \end{bmatrix}$$

where  $\sigma_i = 1, 2, 3, \dots$  are consistent with the operating time. The nonlinear time-varying uncertainty  $\Delta$  is supposed to be

$$\Delta(x, u, t) = 0.1x_1 \sin(x_1 t) + 0.2e^{-2t}u \cos(ut).$$

Let the initial states of the neural state  $P$  and the plant state  $x$  were, respectively,  $P(0) = [0.1]_{3 \times 3}$  and  $x(0) = [0.1 \ 0.1 \ 0.1]^T$ . Learning rates for the neural networks were chosen as  $\eta_p = 8$  and  $\eta_r = 50$ . The upper and lower limits of the nonlinear transformations in (8a) were chosen as  $p_{s,\max} = 2$  and  $\varepsilon = 0.1$ . We used the neural dynamics (9) to solve the Lyapunov matrix inequality (37) at every operating point.

To perform real-time simulation, the neural dynamic equations were discretized using Euler's algorithm and the closed-loop control scheme was experimentally setup on a dSPACE DS1103 digital signal processor (DSP) board. The control board is specifically designed for development of

high-speed multivariable digital controllers and real-time simulations in various fields. It is a complete real-time control system featuring a 333 MHz processor-Motorola PowerPC 604e. For advanced input-output (I/O) purposes, it includes a slave-DSP subsystem based on the Texas Instruments TMS320F240 DSP. The board consists of two DSP's, 2-Mbytes local SRAM, 32-MByte global DRAM, digital I/O channels, and serial interfaces, a timing function to generate interrupts to the DSP, 14-bit D/A and 16-bit A/D converters.

Together with the MathWork's Real-Time Workshop and SIMULINK, a real-time interface program was applied to generate real-time code for SIMULINK off-line models and implement this code on the DSP board. Once we have built our model and downloaded its real-time code to the digital processing hardware we can observe our experiment on the Windows environment.

Real-time simulation results for major signals were illustrated in Figs. 3–5. Fig. 3(a), (b) illustrate, respectively, the convergent behavior of the activation states to the solution matrix  $P$  and the slack matrix  $\tilde{R}_1$ . For clarity, only lower triangular elements of these matrices and their responses within 0 to 5 s were shown. The coefficient matrix  $A$  changes at every 1 s. The solution matrix  $P$  and the slack matrix  $\tilde{R}_1$  reach exact steady-state values (i.e., the matrix values such that (38) holds) less than 0.3 s. Fig. 4(a)–(c) illustrate, respectively, the transient responses of the control gain  $K$ , system state  $x$  and control signal  $u$ . The parameter  $\gamma$  determined by the robust stability condition (39) is illustrated in Fig. 5. It was found from the transient response of  $x$  that, the influence caused by the nonlinear time-varying perturbation has been successfully restrained by the control signal. Clearly, the feedback control system is robustly stabilized. It is

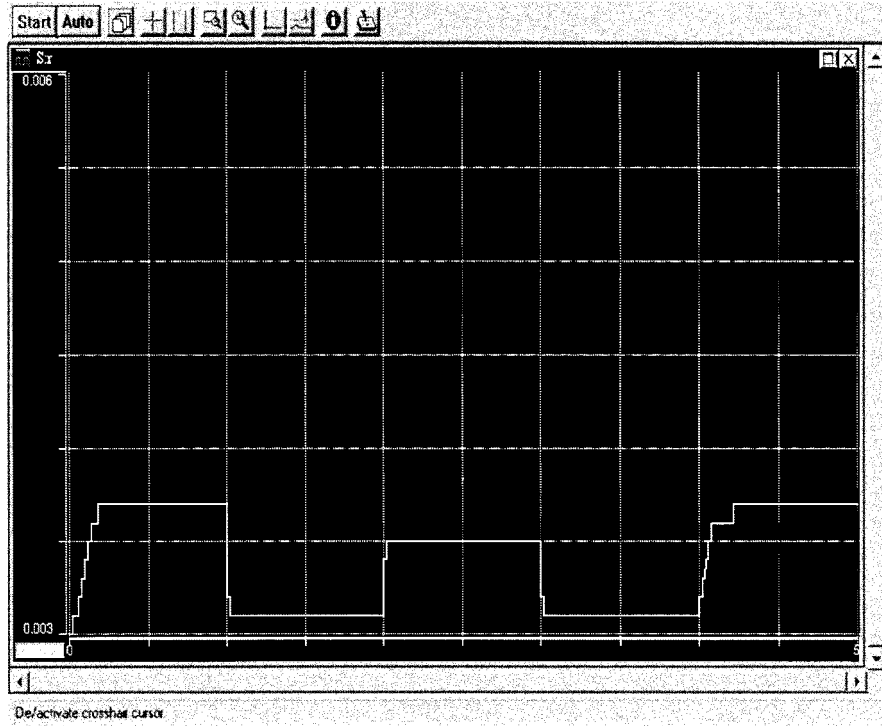


Fig. 5. Change of the design parameter  $\gamma$ .

can also be seen that the proposed neural network solving for the solution of the LMI can follow the slow variation of the system parameters to generate stabilizing control signals. From the results obtained, we can find that the real-time simulation system is quite efficient from the perspective of both running time and also data accuracy.

*Example 2:* Consider the linear time-invariant system (21) with

$$A = \begin{bmatrix} -1 & -1 & -1 \\ -1 & -2 & -1 \\ -1 & -1 & -3 \end{bmatrix}, \quad B = \begin{bmatrix} -1 & 0 & 0 \\ -1 & -1 & 0 \\ -1 & -1 & -1 \end{bmatrix}.$$

The neural dynamics (30) was used to find the solution of the transformed LMI (27). Let  $Q(0) = [0.1]_{3 \times 3}$ ,  $\eta_p = 5000$ ,  $\eta_r = 20\,000$ ,  $p_{\max} = 2$ ,  $\varepsilon = 0.1$ ,  $\sigma = 1$ . Fig. 6(a)–(c) illustrate, respectively, the convergent behavior of the activation states to the solution matrices  $Q$ , the slack matrix  $\tilde{R}$  and the objective matrix  $G$  (only lower triangular elements were shown). The steady states of  $Q$  and  $\tilde{R}$  were approximately obtained as

$$Q = \begin{bmatrix} 0.983\,082 & -0.004\,435\,28 & -0.018\,890\,4 \\ -0.004\,435\,28 & 0.963\,767 & -0.217\,216 \\ -0.018\,890\,4 & -0.217\,216 & 0.295\,135 \end{bmatrix},$$

$$\tilde{R} = \begin{bmatrix} 1.708\,66 & 0 & 0 \\ 1.578\,69 & 1.708\,66 & 0 \\ 1.159\,39 & 0.186\,802 & 1.708\,66 \end{bmatrix}.$$

Clearly, the matrix  $Q$  is symmetric and positive definite, and satisfy the LMI (27). The steady control gain is then obtained as

$$K = \begin{bmatrix} 0.562\,230 & 1.092\,556\,7 & 2.534\,235 \\ 0.052\,782 & 1.080\,935\,2 & 2.493\,074 \\ 0.041\,161 & 0.458\,701 & 2.034\,373 \end{bmatrix}.$$

*Example 3:* Consider the algebraic Riccati matrix inequality (31) with

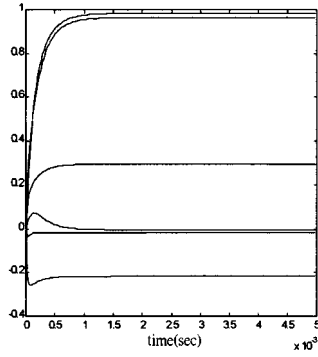
$$A = \begin{bmatrix} -4 & 1 \\ 0 & -7 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 1 \\ 1 & 1 \end{bmatrix}, \quad C = \begin{bmatrix} 0 & 0 \\ 0 & 0 \end{bmatrix},$$

$$D^T + D = \begin{bmatrix} 1 & 0 \\ 0 & 1 \end{bmatrix}, \quad Q = \begin{bmatrix} 1 & 0 \\ 0 & 3 \end{bmatrix}.$$

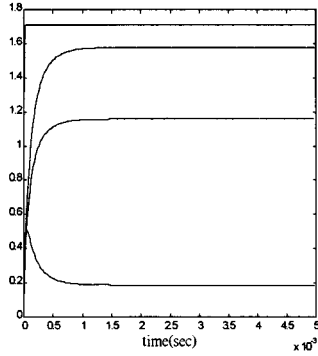
The algebraic Riccati matrix inequality is converted to the LMI (33) with

$$\bar{A} = \begin{bmatrix} 1 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 0 \end{bmatrix}, \quad \bar{B} = \begin{bmatrix} -4 & 1 & 1 & 1 \\ 0 & -7 & 1 & 1 \end{bmatrix}$$

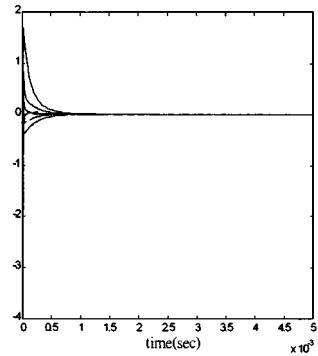
$$\bar{D} = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{bmatrix}.$$



(a)



(b)



(c)

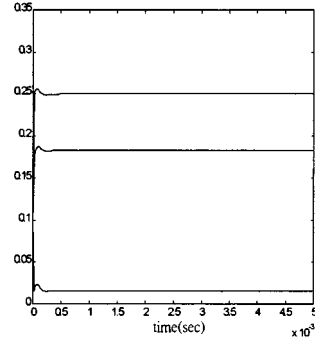
Fig. 6. (a) Transient of  $Q$ , (b) transient of  $\tilde{R}$ , (c) transient of  $G$ .

Let  $P(0) = [0.1]_{2 \times 2}$ ,  $\eta_p = 5000$ ,  $\eta_r = 50000$ ,  $p_{s,\max} = 1$ ,  $\varepsilon = 0.01$ . The neural dynamics (35) was used to solve the algebraic Riccati matrix inequality (31). Fig. 7(a)–(c) illustrates, respectively, the convergent behavior of the activation states to the solution matrix  $P$ , the slack matrix  $\tilde{R}_1$ , and the objective matrix  $G_1$ . The steady states of  $P$  and  $\tilde{R}_1$  were approximately obtained as

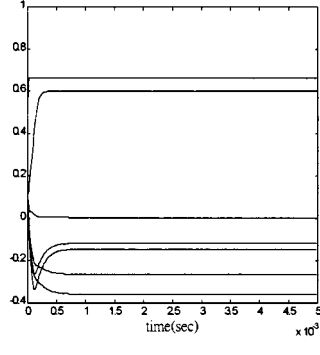
$$P = \begin{bmatrix} 0.183057 & 0.0159725 \\ 0.0159725 & 0.250297 \end{bmatrix}$$

$$\tilde{R}_1 = \begin{bmatrix} 0.66343 & 0 & 0 & 0 \\ 0.00421687 & 0.66343 & 0 & 0 \\ -0.116196 & -0.143594 & 0.66343 & 0 \\ -0.264492 & -0.358474 & 0.601167 & 0.66343 \end{bmatrix}.$$

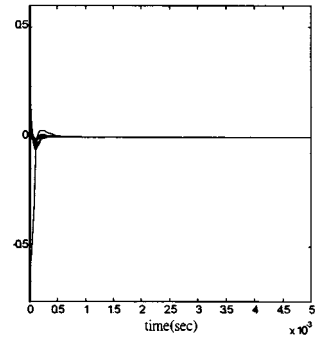
Clearly,  $P$  is symmetric, positive definite, and satisfies the LMI (33).



(a)



(b)



(c)

Fig. 7. (a) Transient of  $P$ , (b) transient of  $\tilde{R}_1$ , and (c) transient of  $G_1$ .

The previous results show the convergence behavior of the solution of a well-conditioned problem, i.e.,  $(\lambda_{\max}(P)/\lambda_{\min}(P)) < 1.7$ . Now consider an alternative system matrix  $A = \begin{bmatrix} -2 & 1 \\ 0 & -31 \end{bmatrix}$  in which  $|a_{22}| \gg |a_{11}|$ . Other matrices and parameters remain invariant. For this case, Fig. 8(a)–(b) illustrate, respectively, the convergence behavior of  $P$  and  $G_1$ . The steady states of both matrices were obtained as

$$P = \begin{bmatrix} 0.368056 & 0.0153872 \\ 0.0153872 & 0.0573461 \end{bmatrix}$$

$$\tilde{R}_1 = \begin{bmatrix} 0.66343 & 0 & 0 & 0 \\ 0.163995 & 0.66343 & 0 & 0 \\ -0.334164 & 0.0314548 & 0.66343 & 0 \\ -0.595741 & 0.280477 & 0.355366 & 0.66343 \end{bmatrix}.$$

Note that in this case we have  $(\lambda_{\max}(P)/\lambda_{\min}(P)) \approx 15.5$

and the matrix  $\tilde{R}_1$  is not diagonally dominant. While the problem is not very well conditioned, the neural dynamic solver still works. For the ill-conditioned problem such as the measure of  $\lambda_{\max}(P)/\lambda_{\min}(P)$  is relatively large, one may use the scaled version of the gradient method [9] to change the coordinate of  $P$  to ensure consistent convergence of the elements of  $P$ .

We now examine the effect of learning rates to the convergence sensitivity. From Fig. 7(a), we can find that the solution matrix  $P$  reaches exact state value less than  $3 \times 10^{-4}$  s. While the different learning rates  $\eta_p = 500$  and  $\eta_r = 5000$  (i.e., 1/10 times to the original setting) were used, the solution matrix  $P$  reaching exact state value increases to  $1.9 \times 10^{-3}$  s. The convergence rate of the former is about six times faster than the latter. Due to nonlinearity of the activation matrix function, there is no quantitative relation between the convergence rate and learning rates that can be explicitly constructed. In general, appropriate learning rates are experimentally chosen by considering hardware limitations of the fabricated neural networks (either using VLSI complimentary metal oxide semiconductor (CMOS) technology or emulated via an advanced DSP control board) and satisfactory convergence performance.

## VI. CONCLUSION

This paper presents a recurrent neural network approach solving for a class of LMI's that are commonly encountered in robust control system analysis and designs. The proposed

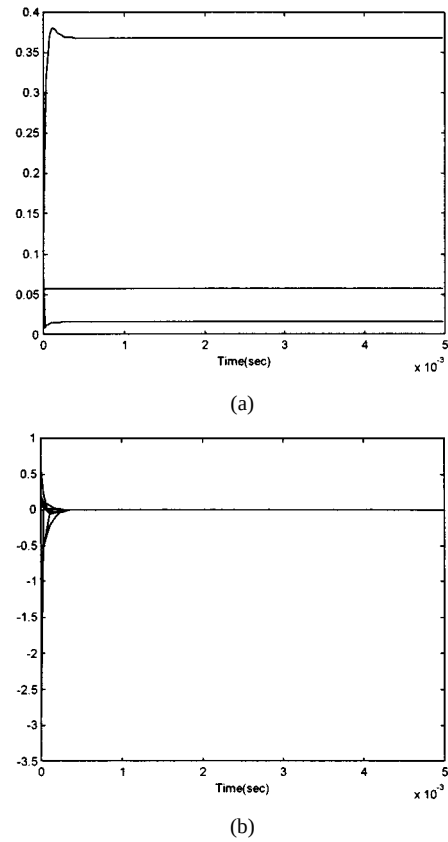


Fig. 8. (a) Transient of  $P$  and (b) transient of  $G_1$ .

$$\begin{aligned}
 \frac{\partial E}{\partial X} &= \begin{bmatrix} \frac{\partial E}{\partial x_{11}} & \cdots & \frac{\partial E}{\partial x_{1n}} \\ \vdots & \ddots & \vdots \\ \frac{\partial E}{\partial x_{m1}} & \cdots & \frac{\partial E}{\partial x_{mn}} \end{bmatrix} = \begin{bmatrix} \sum_{i=1}^l \sum_{j=1}^n f_{ij}(g_{ij}) \frac{\partial g_{ij}}{\partial x_{11}} & \cdots & \sum_{i=1}^l \sum_{j=1}^n f_{ij}(g_{ij}) \frac{\partial g_{ij}}{\partial x_{1n}} \\ \vdots & \ddots & \vdots \\ \sum_{i=1}^l \sum_{j=1}^n f_{ij}(g_{ij}) \frac{\partial g_{ij}}{\partial x_{m1}} & \cdots & \sum_{i=1}^l \sum_{j=1}^n f_{ij}(g_{ij}) \frac{\partial g_{ij}}{\partial x_{mn}} \end{bmatrix} \\
 &= \begin{bmatrix} \sum_{i=1}^l \sum_{j=1}^n f_{ij}(g_{ij}) \frac{\partial \sum_{k=1}^n a_{ik} x_{kj}}{\partial x_{11}} & \cdots & \sum_{i=1}^l \sum_{j=1}^n f_{ij}(g_{ij}) \frac{\partial \sum_{k=1}^n a_{ik} x_{kj}}{\partial x_{1n}} \\ \vdots & \ddots & \vdots \\ \sum_{i=1}^l \sum_{j=1}^n f_{ij}(g_{ij}) \frac{\partial \sum_{k=1}^n a_{ik} x_{kj}}{\partial x_{m1}} & \cdots & \sum_{i=1}^l \sum_{j=1}^n f_{ij}(g_{ij}) \frac{\partial \sum_{k=1}^n a_{ik} x_{kj}}{\partial x_{mn}} \end{bmatrix} \\
 &= \begin{bmatrix} \sum_{i=1}^l a_{i1} f_{i1}(g_{ij}) & \cdots & \sum_{i=1}^l a_{i1} f_{in}(g_{ij}) \\ \vdots & \ddots & \vdots \\ \sum_{i=1}^l a_{in} f_{i1}(g_{ij}) & \cdots & \sum_{i=1}^l a_{in} f_{in}(g_{ij}) \end{bmatrix} \\
 &= A^T F(G)
 \end{aligned}$$

$$\begin{aligned}
\frac{\partial E}{\partial X} &= \begin{bmatrix} \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) \sum_{k=1}^n a_{ki} \left( \sum_{l=1}^n \frac{\partial x_{kl}}{\partial x_{11}} h_{lj} \right) & \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) \sum_{k=1}^n a_{kl} \left( \sum_{l=1}^n \frac{\partial x_{kl}}{\partial x_{12}} h_{lj} \right) & \dots & \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) \sum_{k=1}^n a_{kl} \left( \sum_{l=1}^n \frac{\partial x_{kl}}{\partial x_{1n}} h_{lj} \right) \\ \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) \sum_{k=1}^n a_{ki} \left( \sum_{l=1}^n \frac{\partial x_{kl}}{\partial x_{21}} h_{lj} \right) & \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) \sum_{k=1}^n a_{kl} \left( \sum_{l=1}^n \frac{\partial x_{kl}}{\partial x_{22}} h_{lj} \right) & \dots & \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) \sum_{k=1}^n a_{kl} \left( \sum_{l=1}^n \frac{\partial x_{kl}}{\partial x_{2n}} h_{lj} \right) \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) \sum_{k=1}^n a_{ki} \left( \sum_{l=1}^n \frac{\partial x_{kl}}{\partial x_{n1}} h_{lj} \right) & \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) \sum_{k=1}^n a_{kl} \left( \sum_{l=1}^n \frac{\partial x_{kl}}{\partial x_{n2}} h_{lj} \right) & \dots & \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) \sum_{k=1}^n a_{kl} \left( \sum_{l=1}^n \frac{\partial x_{kl}}{\partial x_{nn}} h_{lj} \right) \end{bmatrix} \\
&= \begin{bmatrix} \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) (a_{1i} h_{1j}) & \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) (a_{1i} h_{2j}) & \dots & \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) (a_{1i} h_{nj}) \\ \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) (a_{2i} h_{1j}) & \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) (a_{2i} h_{2j}) & \dots & \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) (a_{2i} h_{nj}) \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) (a_{ni} h_{1j}) & \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) (a_{ni} h_{2j}) & \dots & \sum_{i=1}^n \sum_{j=1}^n f_{ij}(g_{ij}) (a_{ni} h_{nj}) \end{bmatrix} \\
&= \begin{bmatrix} \sum_{i=1}^n a_{1i} \left( \sum_{j=1}^n f_{ij}(g_{ij}) h_{1j} \right) & \sum_{i=1}^n a_{1i} \left( \sum_{j=1}^n f_{ij}(g_{ij}) h_{2j} \right) & \dots & \sum_{i=1}^n a_{1i} \left( \sum_{j=1}^n f_{ij}(g_{ij}) h_{nj} \right) \\ \sum_{i=1}^n a_{2i} \left( \sum_{j=1}^n f_{ij}(g_{ij}) h_{1j} \right) & \sum_{i=1}^n a_{2i} \left( \sum_{j=1}^n f_{ij}(g_{ij}) h_{2j} \right) & \dots & \sum_{i=1}^n a_{2i} \left( \sum_{j=1}^n f_{ij}(g_{ij}) h_{nj} \right) \\ \vdots & \vdots & \ddots & \vdots \\ \sum_{i=1}^n a_{ni} \left( \sum_{j=1}^n f_{ij}(g_{ij}) h_{1j} \right) & \sum_{i=1}^n a_{ni} \left( \sum_{j=1}^n f_{ij}(g_{ij}) h_{2j} \right) & \dots & \sum_{i=1}^n a_{ni} \left( \sum_{j=1}^n f_{ij}(g_{ij}) h_{nj} \right) \end{bmatrix} \\
&= AFH^T
\end{aligned}$$

networks have been proven to be stable and their steady states are the solution candidate to the LMIs. The solution process can be parallel and distributed in neural computation. It is shown that the developed approach can be extensively extended to solve a class of LMIs. Illustrative examples are provided to demonstrate the operating characteristics of the neural processing.

#### APPENDIX

We only prove 1) and 5), other derivatives can be derived in a similar manner.

Proof of 1): It follows as shown in the matrices at the bottom of the previous page.

Proof of 5): See the matrices shown at the top of the page.

## REFERENCES

- [1] V. Balakrishnan and E. Feron, Eds., "Linear matrix inequalities in control theory and applications," *Int. J. Robust Nonlinear Contr. (Special Issue)*, vol. 6, pp. 869–1099, 1996.
- [2] B. R. Barmish, M. Corless, and G. Leitmann, "A new class of stabilizing controllers for uncertain dynamical systems," *SIAM J. Contr.*, vol. 21, pp. 246–255, 1983.
- [3] S. Boyd, L. E. Ghaoui, E. Feron, and V. Balakrishnan, *Linear Matrix Inequalities in System and Control Theory*. Philadelphia, PA: SIAM, 1994.
- [4] M. Chilali and P. Gahinet, " $H_\infty$  design with pole placement constraints: An LMI approach," in *Proc. 33rd IEEE Conf. Decision Contr.*, 1994, pp. 1511–1516.
- [5] A. Cichocki and R. Unbehauen, *Neural Networks for Optimization and Signal Processing*. New York: Wiley, 1993.
- [6] A. Cichocki, R. Unbehauen, K. Weinzierl, and R. Holzel, "A new neural network for solving linear programming problems," *Europ. J. Operational Res.*, vol. 93, pp. 244–256, 1996.
- [7] P. Gahinet, A. Nemirovskii, A. J. Laub, and M. Chilali, *LMI Control Box: MathWorks*, 1995.
- [8] R. A. Horn and C. R. Johnson, *Matrix Analysis*. New York: Cambridge Univ. Press, 1985.
- [9] P. A. Ioannou and J. Sun, *Robust Adaptive Control*. Englewood Cliffs, NJ: Prentice-Hall, 1996.
- [10] H. Li and M. Fu, "A linear matrix inequality approach to robust  $H_\infty$  filtering," *IEEE Trans. Signal Processing*, vol. 5, pp. 2338–2350, 1997.
- [11] X. Li and C. E. de Souza, "Delay-dependent robust stability and stabilization of uncertain linear delay systems: A linear matrix inequality approach," *IEEE Trans. Automat. Contr.*, vol. 42, pp. 1144–1148, 1997.
- [12] S. Limanond and J. Si, "Neural-network-based control design: An LMI approach," *IEEE Trans. Neural Networks*, vol. 9, pp. 1422–1429, 1998.
- [13] C. L. Lin, "Synthesis of real-time robust flexible structure controllers," *AIAA J. Guid., Contr., Dyna.*, vol. 21, pp. 1001–1004, 1998.
- [14] S. Sivrioglu and K. Nonami, "LMI approach to gain scheduled  $H_\infty$  control beyond PID control for gyroscopic rotor-magnetic bearing system," in *Proc. 35th IEEE Conf. Decision Contr.*, 1996, pp. 3694–3699.
- [15] G. W. Stewart, *Introduction to Matrix Computation*. New York: Academic, 1973.
- [16] L. Vandenberghe and V. Balakrishnan, "Algorithms and software for LMI problems in control," *IEEE Contr. Syst. Mag.*, vol. 17, pp. 89–95, 1997.
- [17] J. Wang, "Recurrent neural networks for solving linear matrix equalities," *Comput. Math. Applic.*, vol. 26, pp. 23–34, 1993.
- [18] J. Wang and G. Wu, "A multilayer recurrent neural network for solving continuous-time algebraic Riccati equation," *Neural Networks*, vol. 11, pp. 935–950, 1998.
- [19] H. O. Wang, K. Tanaka, and M. Griffin, "An approach to control of nonlinear systems: Stability and design issues," *IEEE Trans. Fuzzy Syst.*, pp. 14–23, 1996.
- [20] L. X. Wang and J. M. Mendel, "Three-dimensional structured networks for matrix equation solving," *IEEE Trans. Comput.*, vol. 40, pp. 1337–1346, 1991.
- [21] Y. Xia, "A new neural network for solving linear and quadratic programming problems," *IEEE Trans. Neural Networks*, vol. 7, pp. 1544–1547, 1996.
- [22] J. M. Zurada, *Introduction to Artificial Neural Systems*. St. Paul, MN: West, 1992.



**Chun-Liang Lin** was born in Tainan, Taiwan, R.O.C., in 1958. He received the Ph.D. degree in aeronautical and astronautical engineering from the National Cheng Kung University, Taiwan, in 1991.

He is currently an Associate Professor in the Department of Automatic Control Engineering at Feng Chia University, Taichung, Taiwan, R.O.C. His research interests include guidance and control, intelligent control, and robust control.

**Chi-Chih Lai** was born in Zhanhua, Taiwan, R.O.C., in 1949. He received the B.S. degree in mechanical engineering in 1973 and the M.S. degree in industrial engineering in 1975 both from the Feng Chia University, Taiwan, R.O.C.

He is currently an Associate Professor in the Department of Automatic Control Engineering at Feng Chia University, Taichung, Taiwan. His research interests include sensing element design and measurement technology.



**Teng-Hsien Huang** was born in Taiwan, R.O.C., in 1970. He received the B.S. and M.S. degrees in automatic control engineering from the Feng Chia University, Taichung, Taiwan, R.O.C., in 1996 and 1999, respectively.

His research interests are the design of VLSI circuits and artificial neural networks.